Name : Gunjan Chandnani **Roll No** : 13 **Div** : D20A

EXPERIMENT 5

AIM : Deploying a Voting/Ballot Smart Contract

THEORY:

Q1: What is the relevance of `require` statements in Solidity functions?

In Solidity, the `require` statement works like a **security checkpoint** inside a function. It checks whether a condition is true before allowing the rest of the code to run. If the condition is false, the function immediately stops and all changes made during that transaction are **reverted (undone)**. This prevents invalid or unauthorized actions from affecting the blockchain data.

In simple words, `require` protects the smart contract from wrong inputs or misuse.

Common Uses in a Voting Smart Contract:

- Checking if the user has the right to vote.
  `require(voters[msg.sender].weight > 0, "Has no right to vote");`
- Ensuring a voter has not already voted.
- Allowing only the chairperson to give voting rights.

Why it is Important:

- Increases **security** of the contract.
- Maintains **correctness** of data.
- Prevents invalid transactions.
- Shows helpful **error messages** to users and developers.

Q2: Understanding `mapping`, `storage`, and `memory`

1. Mapping

A `mapping` in Solidity is like a **dictionary or hash table** where each key is connected to a value.

Syntax:

```
mapping(keyType => valueType)
```

Example:

```
mapping(address => Voter) public voters;
```

Here, every Ethereum address is linked to a `Voter` structure containing their details.

Key Points:

- Very fast lookup.
- Gas-efficient for searching data.
- Does **not** have length.
- Cannot be directly looped or iterated.

Mappings are useful when you need to store user-specific information such as voting status.

---

2. Storage

`storage` refers to **permanent memory** on the blockchain.

Characteristics:

- Data is saved forever unless changed.
- Expensive in terms of gas cost.
- Used for contract-level variables.
- Suitable for important data like voter records or proposal counts.

Example: Information stored in `voters` mapping remains available throughout the contract's lifetime.

---

3. Memory

`memory` is **temporary memory** used only during a function call.

Characteristics:

- Data is deleted after the function finishes.
- Cheaper than storage.

- Used for temporary calculations, inputs, or string handling.

Example: Proposal names passed into the constructor are usually stored in memory before being saved permanently.

Conclusion:
A good smart contract balances `storage` and `memory` to reduce gas cost while keeping essential data safe.

Q3: Why use `bytes32` instead of `string`?

Earlier smart contracts often used `bytes32` instead of `string` mainly for **performance and gas efficiency**.

bytes32

- Fixed size (exactly 32 bytes).
- Faster comparison.
- Lower gas cost.
- Limited to 32 characters.
- Not very user-friendly for long names.

string

- Dynamic size (can store long text).
- Easier for users and developers.
- More readable.
- Higher gas consumption.
- Slightly slower processing.

Modern Practice:
Developers now prefer `string` for better readability and user experience, especially in applications like voting systems where names should be clear. However, `bytes32` is still chosen when gas savings and speed are top priority.

Summary:

- Use **bytes32** → when efficiency and cost matter most.
- Use **string** → when readability and flexibility are more important.

CODE:

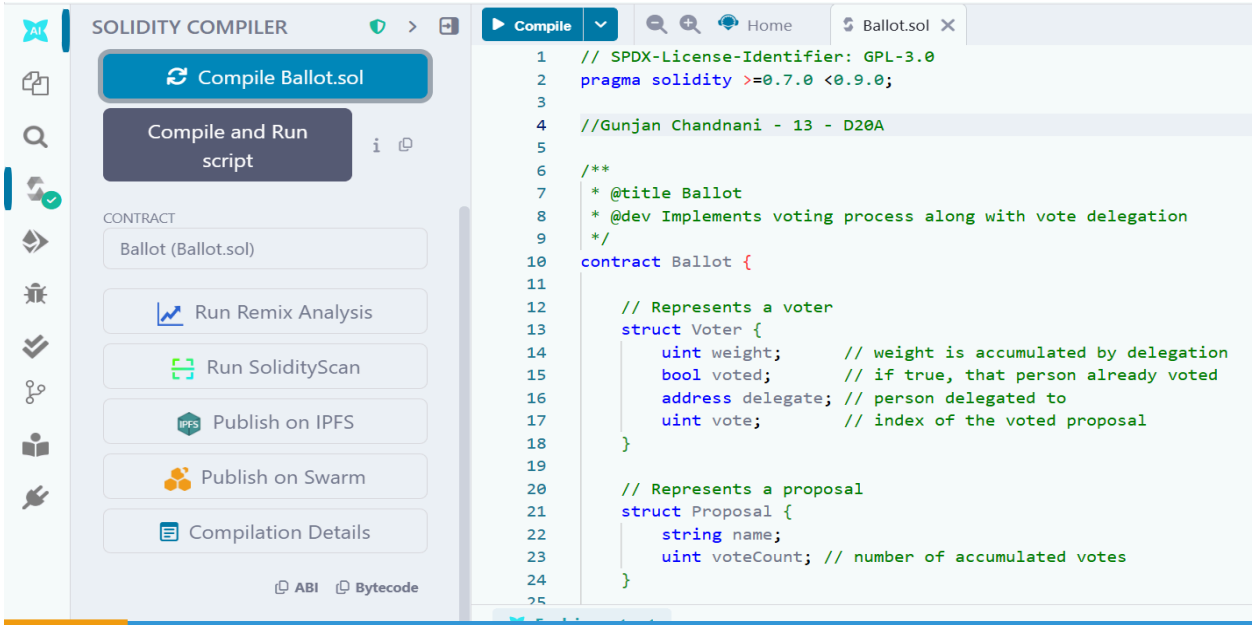
```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {

    // Represents a voter
    struct Voter {
        uint weight;      // weight is accumulated by delegation
        bool voted;       // if true, that person already voted
        address delegate; // person delegated to
        uint vote;        // index of the voted proposal
    }

    // Represents a proposal
    struct Proposal {
        string name;
        uint voteCount; // number of accumulated votes
    }

    address public chairperson;

    // Stores voter details
    mapping(address => Voter) public voters;

    // Array of proposals
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot with proposal names
     */
    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        for (uint i = 0; i < proposalNames.length; i++) {
```

```
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}

/**
 * @dev Give voting right to a voter
 */
function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote.");
    require(!voters[voter].voted, "The voter already voted.");
    require(voters[voter].weight == 0, "Voter already has the right to vote.");

    voters[voter].weight = 1;
}

/**
 * @dev Delegate vote to another voter
 */
function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "You have no right to vote");
    require(!sender.voted, "You already voted.");
    require(to != msg.sender, "Self-delegation is disallowed.");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Found loop in delegation.");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right to vote");

    sender.voted = true;
    sender.delegate = to;

    if (delegate_.voted) {
```

```
        proposals[delegate_.vote].voteCount += sender.weight;
    } else {
        delegate_.weight += sender.weight;
    }
}

/**
 * @dev Vote for a proposal
 */
function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "Has no right to vote");
    require(!sender.voted, "Already voted.");
    require(proposal < proposals.length, "Invalid proposal");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;
}

/**
 * @dev Compute winning proposal index
 */
function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

/**
 * @dev Get winner name
 */
function winnerName() external view returns (string memory winnerName_) {
```

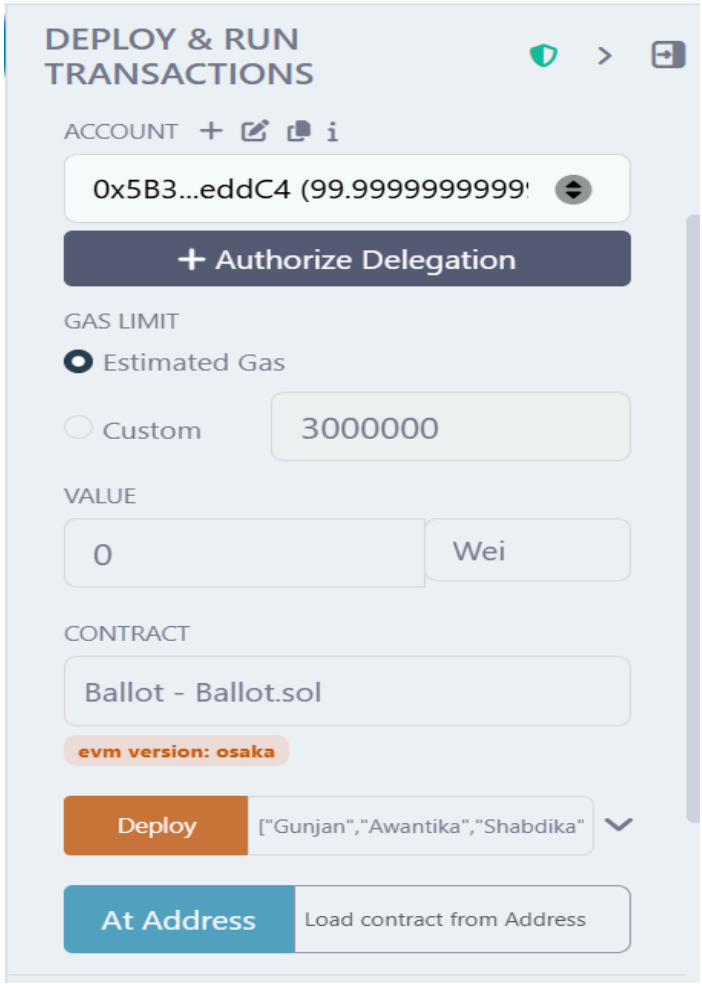
```
        winnerName_ = proposals[winningProposal()].name;
    }
}
```

OUTPUT :



Step 1:

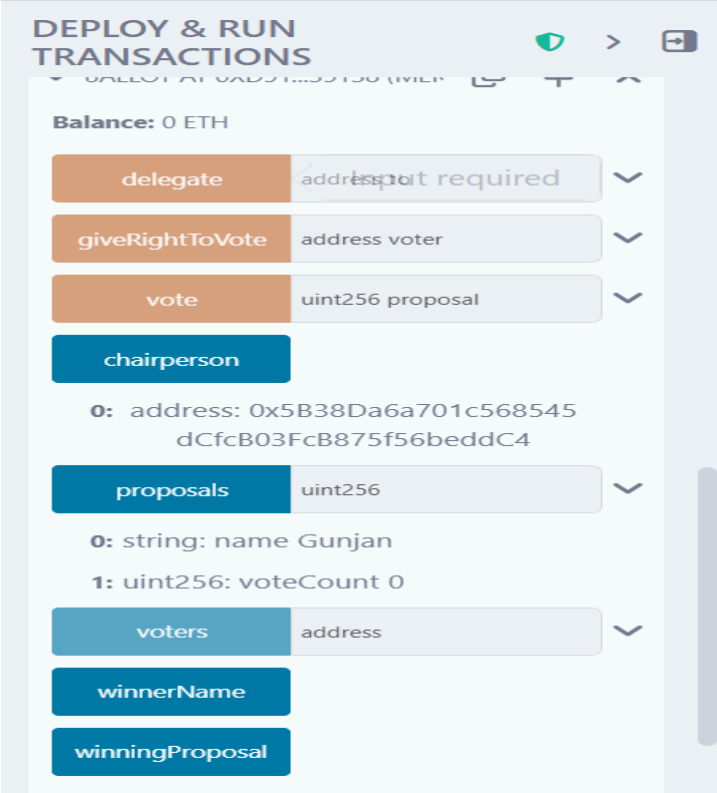
Deploy Contract 3 proposals are created “Gunjan””, “Awantika”” and “Shabdika”” all with vote count 0



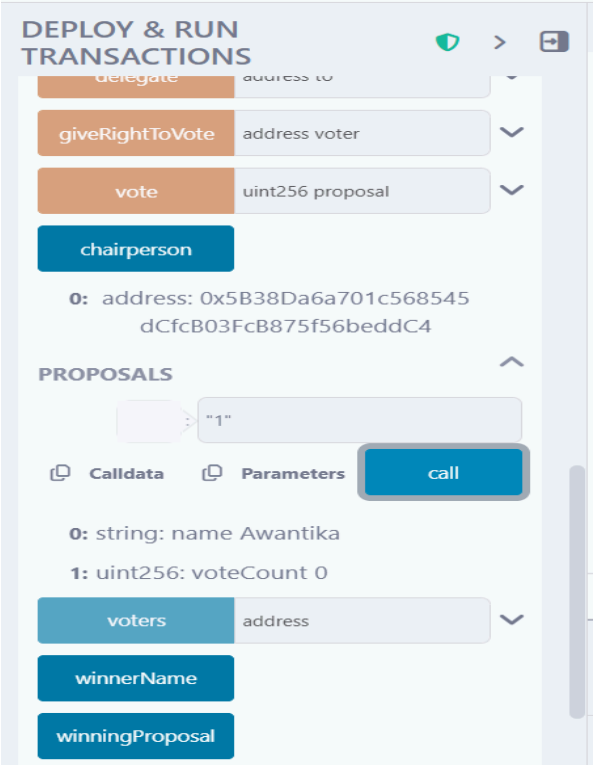
Step 2: Check Chairperson
Here, Account 1 - **"Gunjan"** is the chairperson

Step 3: Check proposals

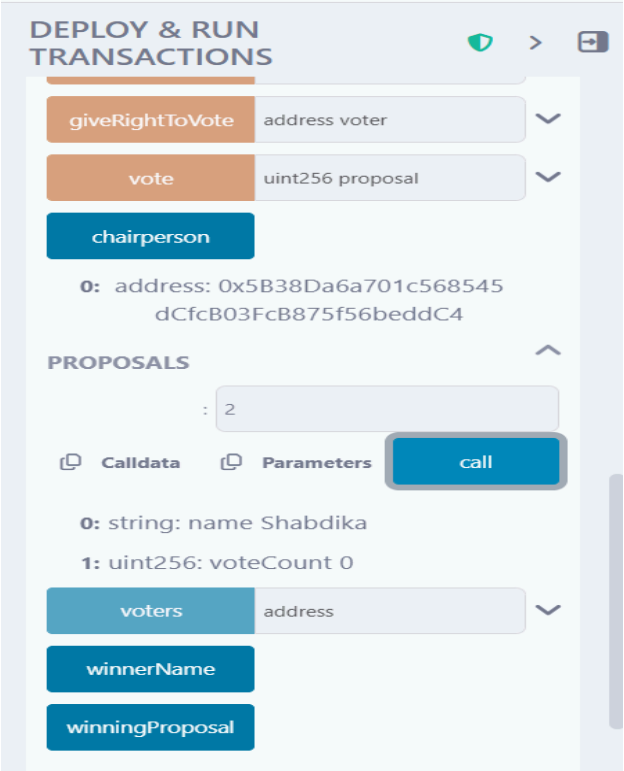
For Account 1- “Gunjan”



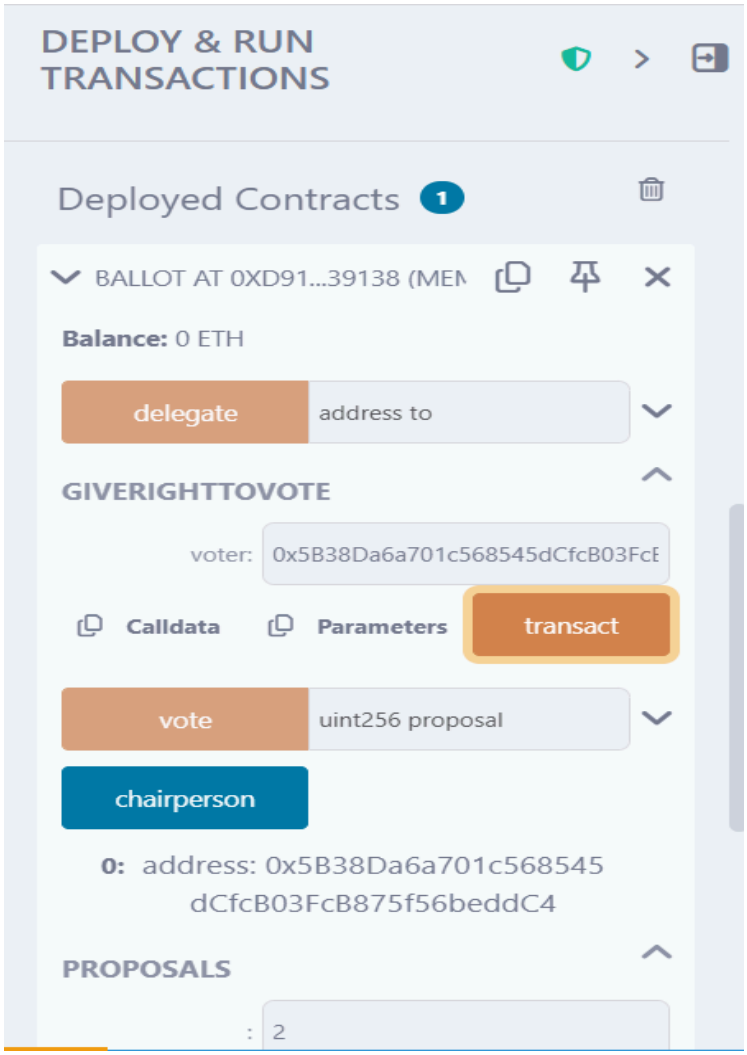
For Account 2 - “”Awantika”



For Account 3 - “Shabdika”

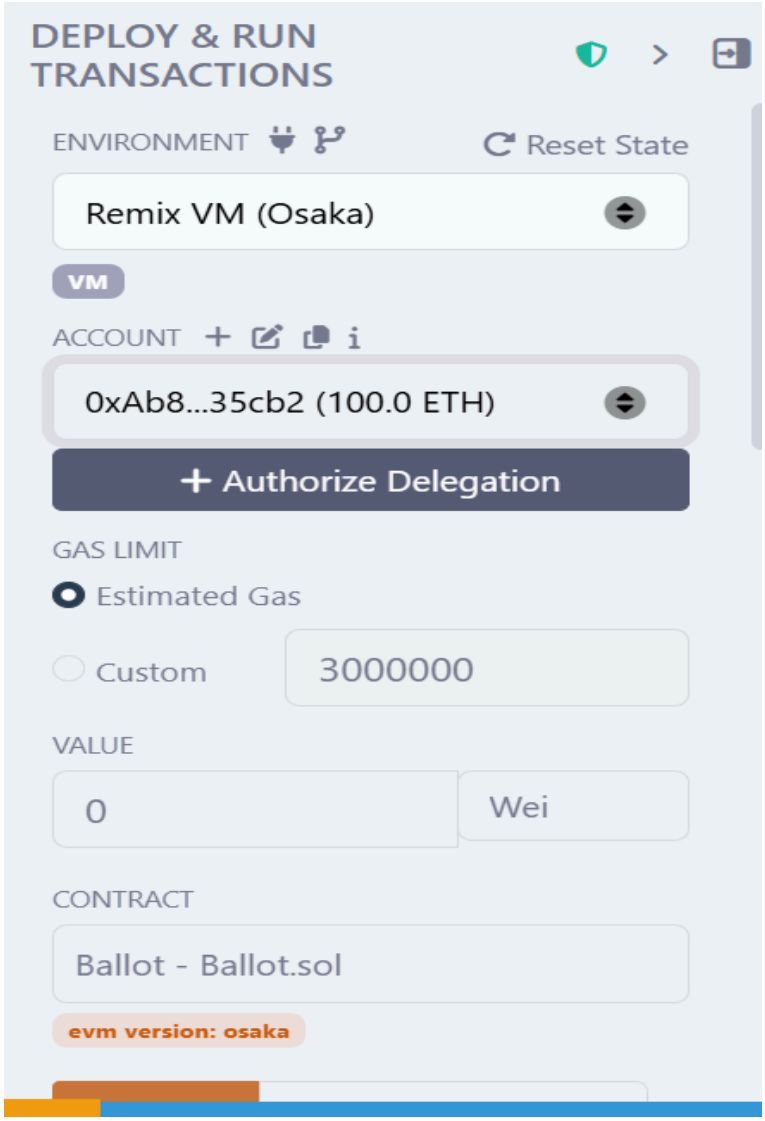


Step 4: Give voting right to another
account Keep account 1 - Gunjan selected and in **giveRightToVote(address voter)** paste the address of Account 2 - “Awantika” and click on the transact button.

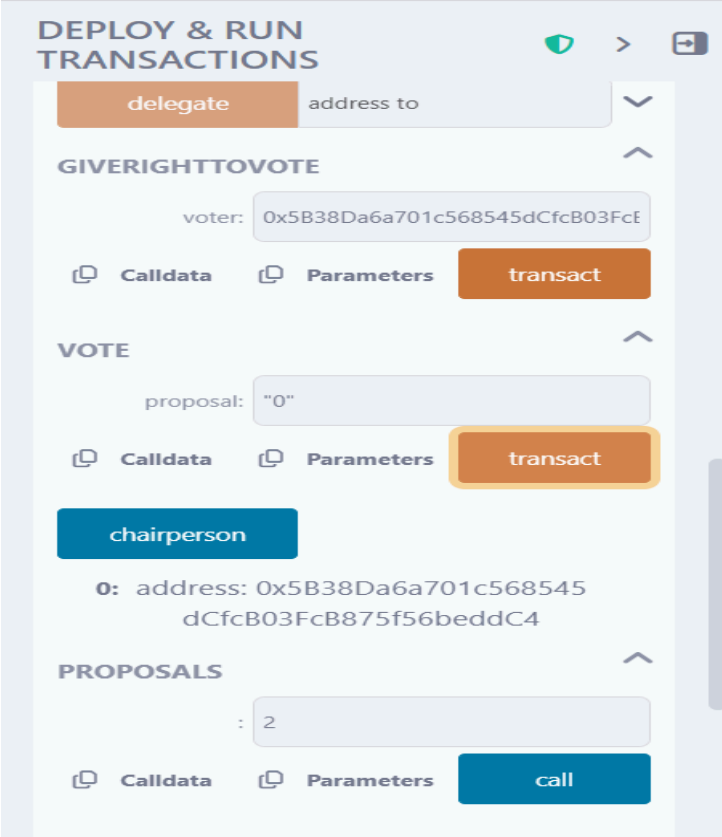


Step 5: Switch to Account 2

From Account dropdown select **Account 2 - “Awantika”**

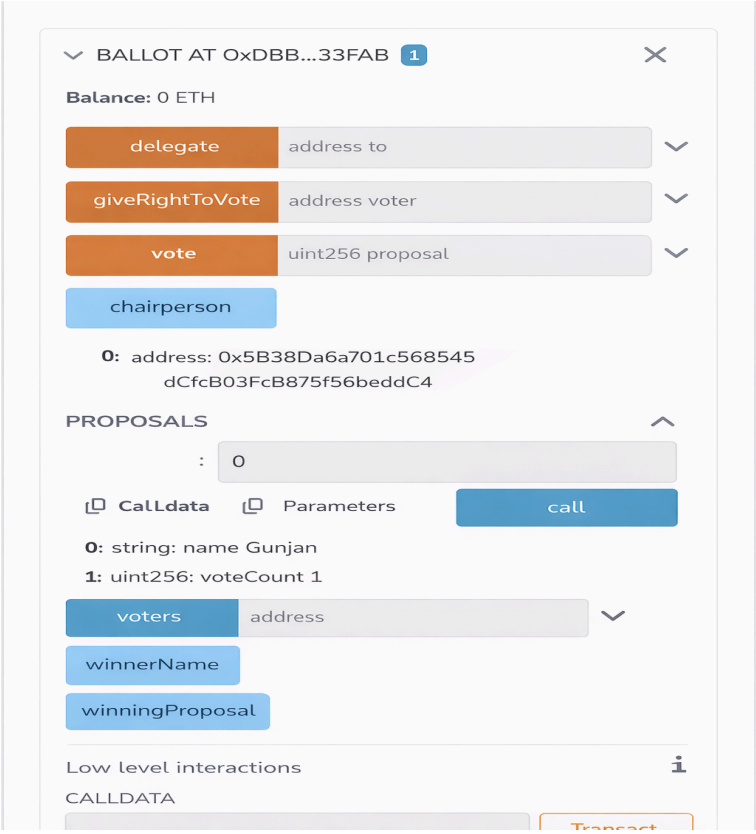


Now, vote for Account 1 - “Gunjan” by vote(0).



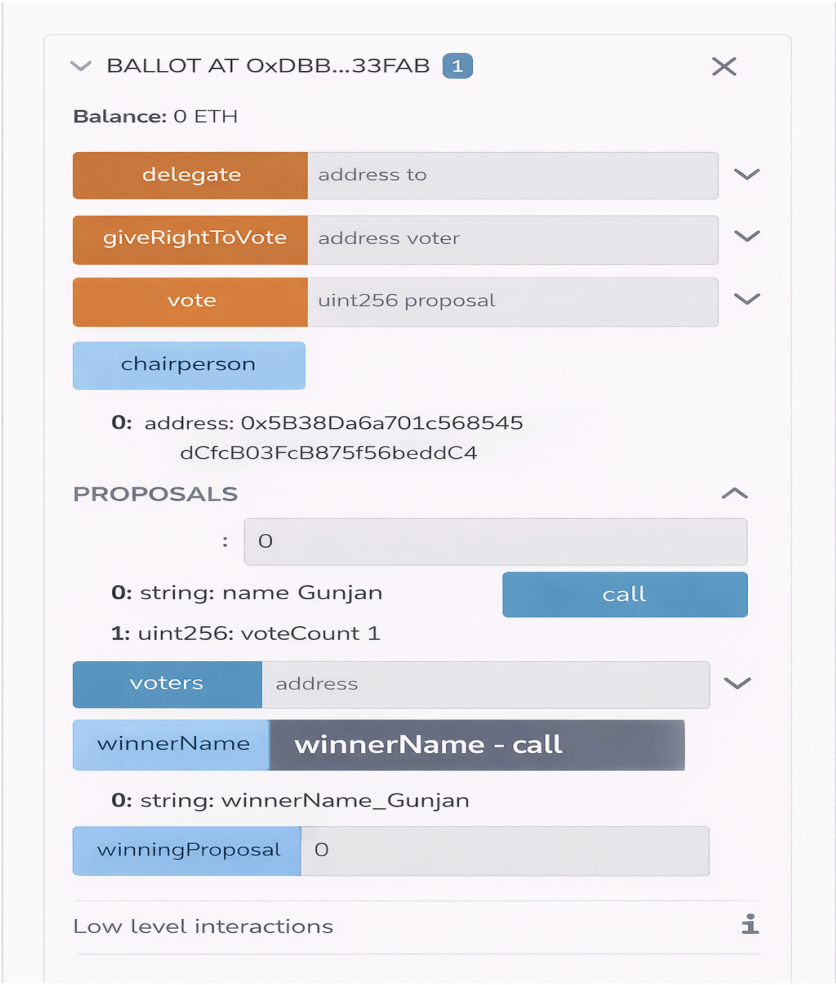
Step 6: Check Proposal

Here, the vote count for Account 1 - **"Gunjan"** increases.



Step 7: Check winner

Here, Account 1 - **"Gunjan"** is the winner as it has the highest votes.



<u>CONCLUSION:</u>

Through this experiment, the Voting/Ballot smart contract was successfully deployed using Solidity in the Remix IDE. Important concepts such as require statements, mapping, and data locations like storage and memory were understood while performing the contract execution. The difference between using bytes32 and string for proposal names was also studied, which helped in understanding gas efficiency and readability. Overall, this experiment helped in gaining practical knowledge about designing and deploying a voting smart contract on the blockchain.